

Verteilte Systeme – Übungsblatt 2

Schnappschuss eines konsistenten globalen Zustands

Bericht

Sommersemester 2026

1 Problemstellung und Entwurf

Wir bauen ein kleines verteiltes „Bankhaus“: n Prozesse P_0 bis P_{n-1} in **sim4da-S26**, vollvernetzt, jeder mit Startguthaben 1000. Jeder Prozess überweist in unregelmäßigen Abständen einen zufälligen Betrag an einen zufälligen anderen Prozess, bucht dabei **sofort** ab und verschickt die Überweisung mit spürbarer, zufälliger **Latenz** – und ganz bewusst **ohne FIFO**, sodass Nachrichten einander unterwegs überholen können. Über dieses System legen wir wiederholt einen **konsistenten Schnitt**: eine Momentaufnahme, die – obwohl das Geld die ganze Zeit weiter durchs Netz fließt – exakt die Gesamtsumme $S = n \cdot 1000$ reproduziert, weil sie sowohl alle Kontostände *als auch* alle gerade unterwegs befindlichen Überweisungen erfasst. Konkret behandelt dieser Bericht:

- **Aufgabe 1** – die Basisanwendung selbst: sofortige Abbuchung, zufällige Latenz, FIFO explizit abgeschaltet.
- **Aufgabe 2** – das **Einfärbeverfahren (Variante a)** als Schnappschuss-Algorithmus, mitsamt der beiden Sonderfälle, die Nicht-FIFO-Zustellung mit sich bringt, und einer Terminierung ganz ohne die zweite Chandy-Lamport-Marke.
- **Aufgabe 3** – der Beweis, dass der eingefärbte Schnitt konsistent ist, ein naiver Vergleichsschnitt, der es absichtlich *nicht* ist, sowie eine kleine Kosten-Statistik.
- Eine **interaktive Visualisierung** (<visuals/index.html>), die genau dieses Wettrennen zwischen Überweisungen und Schnappschuss im Browser sichtbar macht.

Zentrale Entwurfsentscheidung. Wir haben uns bewusst für **Variante (a)**, das **Koordinator-/ Einfärbeverfahren**, entschieden statt für Chandy-Lamport, und zwar aus einem einzigen, aber entscheidenden Grund: Chandy-Lamport **setzt FIFO-Kanäle voraus**. Der Marker trennt auf jedem Kanal alte von neuen Nachrichten nur deshalb zuverlässig, weil er selbst nie von einer älteren Nachricht überholt werden kann. Genau das ist in unserem Setup (Aufgabenstellung: FIFO ausdrücklich abgeschaltet) nicht gegeben – eine alte, weiße Überweisung könnte den Marker jederzeit überholen, und die Kanalaufzeichnung „Marker bis Marker“ bräche in sich zusammen. Das Einfärbeverfahren umgeht dieses Problem elegant: Statt sich auf die *Ankunftsreihenfolge* zu verlassen, entscheidet allein die **Farbe** einer Nachricht (weiß oder schwarz), ob sie zum Schnitt gehört – und diese Farbe steht schon beim Absenden fest, unabhängig davon, wann und in welcher Reihenfolge sie ankommt.

2 Aufgabe 1 – Basisanwendung

Jeder Prozess P_i (**BankNode**) startet mit Saldo 1000, wählt zufällig Empfänger und Betrag $0 < b \leq \text{balance}_i$, **bucht sofort ab** und sendet **Transfer(b)** als **record** mit Marker-Interface **Message**; der Empfänger schreibt beim Empfang gut. Zwischen Abbuchen und Senden liegt eine zufällige **Latenz** (20–120 ms per **sleep**), sodass Geld merklich lange „unterwegs“ ist – genau dieses Zeitfenster macht den Schnitt später überhaupt interessant.

FIFO ist abgeschaltet , wörtlich wie im Blatt gefordert:

```
SimulationBehavior.setMessageQueueSelectionDistributionFunction(
```

```
RandomValues.getUniformDistribution());
```

Empfänger entnehmen ihrer Warteschlange also eine zufällige Nachricht – Nachrichten überholen einander. Da `receive()` blockiert, ist die Anwendung nachrichtengetrieben: Jeder Prozess stößt einige Start-Überweisungen an, jede empfangene Überweisung löst (bis zu einem Budget) die nächste aus; `simulator.simulate(seconds)` begrenzt die Laufzeit zusätzlich von außen.

3 Aufgabe 2 – Einfärbeverfahren unter Nicht-FIFO

Ablauf. Die Farbe ist zu einem **Rundenzähler** verallgemeinert (weiß = Runde $r-1$, schwarz = r), sodass beliebig viele Schnappschüsse nacheinander möglich sind, ohne den Prozess neu starten zu müssen. Jede **Transfer**-Nachricht trägt die Runde des Senders **zum Zeitpunkt des Abbuchens** als Stempel. Der Koordinator P0 (selbst ein ganz normaler Bankprozess, kein separater Beobachter) sichert zuerst den eigenen Zustand und multicastet **StateRequest(r)** („state?, Farbe“). Jeder Empfänger färbt sich um, sichert seinen Saldo und meldet ihn per **StateReport**. Interessant wird es bei den beiden Fällen, die erst durch die fehlende FIFO-Ordnung überhaupt möglich werden (**BankNode.onTransfer**):

1. **Nachträgliche schwarze Nachricht** (Stempel $>$ eigene Runde, „Nachricht aus der Zukunft“): Ein Prozess kann eine schwarze Überweisung erhalten, bevor ihn selbst überhaupt ein **StateRequest** erreicht hat – weil sie ihn einfach überholt hat. Er färbt sich dann **zuerst** um und sichert seinen Zustand, erst danach wird die Nachricht verbucht. Abbuchung und Gutschrift liegen so beide sauber hinter dem Schnitt – der Betrag zählt korrekt weder zu den Konten noch zu den Kanälen.
2. **Schwarzer Prozess empfängt weiße Nachricht** (Stempel $<$ Runde): Diese Überweisung war beim Schnitt unterwegs und gehört damit zum **Kanalzustand**. Wir haben uns dafür entschieden, dass der Empfänger eine Kopie als **ChannelReport(from, to, b)** an den Koordinator weiterleitet und danach ganz normal verbucht. *Warum nicht anders?* Die naheliegende Alternative – den Betrag nachträglich in den bereits gemeldeten Zustand einzurechnen – würde korrigierende Zweitmeldungen erfordern und Prozess- und Kanalzustand munter vermischen. Die Weiterleitung hält beides sauber getrennt und erlaubt uns außerdem, die Kanalinhalte pro Kanal $P_i \rightarrow P_j$ explizit auszugeben, wie es die Aufgabe verlangt.

Terminierung ohne FIFO – Defizitzählung. Unter FIFO würde die zweite Markernachricht die Kanalaufzeichnung sauber beenden; das entfällt hier, also brauchten wir einen anderen Trick. Jeder Prozess zählt gesendete und empfangene Basisnachrichten **getrennt pro Stempel**; das **StateReport** für Runde r meldet dem Koordinator die Zähler für Stempel $r-1$ gleich mit. Der Koordinator weiß dadurch exakt, wie viele weiße Nachrichten beim Schnitt noch unterwegs sein müssen:

$$D = \sum_i sent_i(r-1) - \sum_i recv_i(r-1).$$

Er wartet also nicht auf ein Signal „Kanal fertig“, sondern einfach so lange, bis genau D **ChannelReports** eingetroffen sind – erst dann gibt er den vollständigen globalen Zustand aus (alle Konten, alle Kanalinhalte, Summe, Konsistenzurteil). Das Schöne daran: Diese Zählung ist komplett unabhängig von der Ankunftsreihenfolge und damit von Natur aus nicht-FIFO-fest.

Annahmen (wie in der Vorlesung): keine Nachrichtenverluste (sonst würde die Defizitzählung nie fertig), keine Prozessabstürze; die Kanäle sind zuverlässig, aber beliebig langsam und umsortierend; Runden laufen sequentiell, eine neue startet erst nach Abschluss der vorigen – dadurch sind bei Rundenstart garantiert nur Nachrichten mit Stempel $r-1$ unterwegs.

Abbildung 1 fasst das gesamte Verfahren an einem kleinen Beispiel mit $n = 3$ zusammen – beide oben beschriebenen Sonderfälle treten hier gemeinsam auf: Die schwarze **Transfer(50)** überholt den **StateRequest** (P1 muss sich vorher umfärben), und die weiße **Transfer(200)** erreicht das bereits schwarze P2 und landet als **ChannelReport** im Kanalzustand. Trotz dieses Durcheinanders erfasst der Schnitt am Ende exakt $S = 3000$.

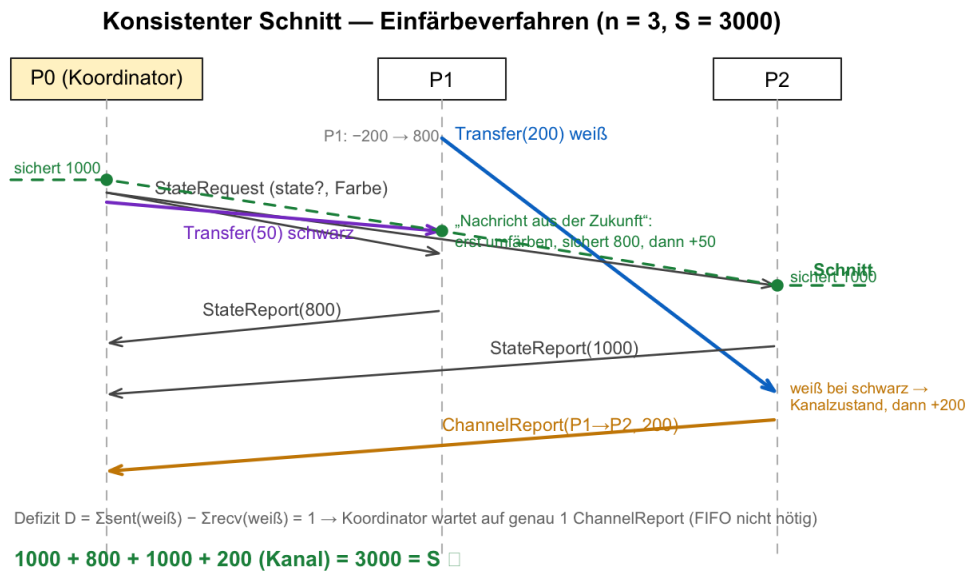


Abbildung 1: Konsistenter Schnitt des Einfärbeverfahrens ($n=3, S=3000$) mit beiden Sonderfällen. Die schwarze Nachricht **Transfer(50)** überholt den **StateRequest** – P1 färbt sich deshalb vor dem Verbuchen um. Die weiße **Transfer(200)** wurde ganz zu Beginn (noch weiß) losgeschickt und erreicht das inzwischen schwarze P2 erst viel später; sie wird dort korrekt als **ChannelReport** an den Koordinator nachgemeldet. Die grüne Linie ist der Schnitt; die erfasste Summe ist $1000+800+1000+200 = 3000 = S$ (Abbildung mit R erstellt).

4 Aufgabe 3 – Konsistenznachweis und Statistik

Experimente. `BankhausTest.coloredSnapshotIsAlwaysConsistent` löst mit $n = 6$ mehrere Schnappschüsse **im laufenden Betrieb** aus – die Überweisungen werden dafür nicht angehalten. In jeder einzelnen Runde ergibt die erfasste Summe exakt S , regelmäßig mit einem spürbaren Anteil des Geldes in den Kanälen statt auf Konten – ein schöner Beleg dafür, dass der Schnitt wirklich mitten im Betrieb liegt und nicht etwa nur zufällig in einer ruhigen Phase. `naiveSnapshotYieldsInconsistencies` zeigt mit $n = 8$ das Gegenstück: Der naive Schnappschuss (`BalanceRequest/BalanceReply`, Kanäle werden schlicht ignoriert) liefert regelmäßig Summen $\neq S$.

Warum der naive Schnitt scheitert. Ein Schnitt ist konsistent, wenn für je zwei Ereignisse gilt: Liegt ein Ereignis e_2 im Schnitt und geht ihm kausal ein Ereignis e_1 voraus ($e_1 <_k e_2$), dann muss auch e_1 im Schnitt liegen. Für unser Bankhaus heißt das ganz konkret: Ist die Gutschrift e_2 erfasst, muss die kausal vorausgehende Abbuchung e_1 es auch sein. Der naive Schnappschuss liest die Konten aber zu völlig unkoordinierten Zeitpunkten ab: Antwortet P_1 *vor* seiner Abbuchung und P_2 *nach* seiner Gutschrift, landet e_2 im Schnitt, e_1 aber nicht – der überwiesene Betrag zählt doppelt („Geld entsteht aus dem Nichts“). Symmetrisch „verschwindet“ Geld, wenn umgekehrt e_1 erfasst wird, e_2 aber nicht, und der unterwegs befindliche Betrag einfach unter den Tisch fällt. Das Einfärbeverfahren schließt beide Fälle konstruktionsbedingt aus: Die Farbe macht die Schnittzugehörigkeit jedes einzelnen Ereignisses eindeutig, und weiße Nachrichten an bereits schwarze Prozesse werden explizit als Kanalzustand nachgereicht statt verschluckt.

Wie der eingefärbte Schnitt genau denselben Fall korrekt behandelt, zeigt Abbildung 1 weiter oben; Abbildung 2 stellt daneben, wie der naive Schnitt an genau dieser Situation scheitert.

Statistik. `statisticsExperiment` variiert $n \in \{4, 8, 16\}$ und die Überweisungsfrequenz (zwei Latenzprofile) und schreibt pro Schnappschuss eine Zeile nach `stats.csv`. Die Kontrollnachrichten

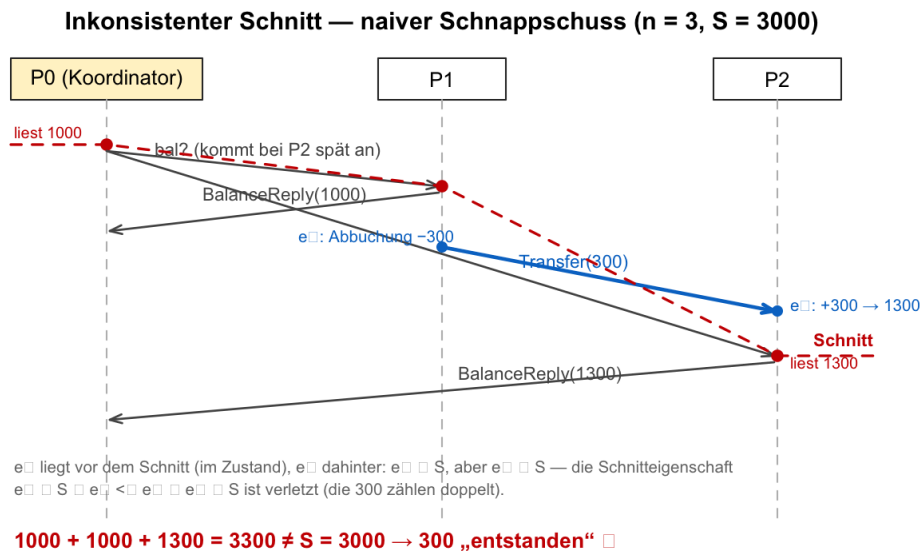


Abbildung 2: Inkonsistenter Schnitt des naiven Schnappschusses an derselben Situation wie Abbildung 1: P1 antwortet noch vor der Abbuchung e_1 , P2 aber erst nach der Gutschrift e_2 – der Schnitt enthält e_2 ohne e_1 und verletzt damit die Schnitteigenschaft; die 300 zählen doppelt, die erfasste Summe ist $3300 \neq S$. (Abbildung mit R erstellt)

pro eingefärbtem Schnappschuss lassen sich sauber in eine Formel fassen:

$$K(n, E) = 2(n - 1) + E$$

($n-1$ Requests, $n-1$ Reports, E ChannelReports; die lokalen Meldungen des Koordinators an sich selbst kosten nichts). Genau daran zeigen sich die aus der Vorlesung bekannten Nachteile des Verfahrens ganz praktisch:

- **Hohes Nachrichtenaufkommen.** Pro Schnappschuss kommen $\mathcal{O}(n + E)$ Zusatznachrichten hinzu, wobei E mit Frequenz und Latenz der Überweisungen wächst. Bei häufigen Schnappschüssen dominiert irgendwann der Kontrollverkehr die eigentliche Nutzlast.
- **„Nicht verteilt genug“.** P0 ist Engpass und Single Point of Failure zugleich: Alle Meldungen laufen bei ihm zusammen, nur er kennt am Ende den vollständigen globalen Zustand, und fällt er aus, terminiert der Schnappschuss nie – weshalb wir die Annahme „keine Abstürze“ überhaupt brauchen.

5 Visualisierung

Neben dem eigentlichen Java-Code liegt `visuals/index.html` bei – eine kleine, abhängigkeitsfreie Miniatur-Nachbildung der Implementierung, die direkt im Browser läuft. Sie zeigt n Prozesse in einem vollvernetzten Ring, die sich fortlaufend Beträge überweisen: sichtbar mit Latenz (Nachrichten brauchen echte Zeit, um anzukommen) und ohne FIFO (sie können sich dabei überholen). Per Knopfdruck lässt sich entweder ein eingefärbter oder ein naiver Schnappschuss auslösen, während im Hintergrund weiter überwiesen wird – genau wie in den JUnit-Experimenten aus Aufgabe 3.

Das Herzstück ist das **Zeitdiagramm** unten im Bild (Abbildung 3): Es zeichnet live mit, welche Nachricht wann von wem zu wem unterwegs ist, und zieht den **Schnitt als Linie durch die lokalen Zustandssicherungen** – beim Einfärbverfahren bleibt diese Linie sauber und die Summe stimmt stets mit S überein; beim naiven Schnappschuss sieht man mit bloßem Auge, wie eine Überweisung die Linie rückwärts kreuzt und die Summe kippt. Die Idee dahinter: Was in Abbildung 1 und 2 als statisches Beispiel gezeigt wird, kann man sich hier für beliebige viele

Knoten und beliebig viele Durchläufe selbst „erspielen“.

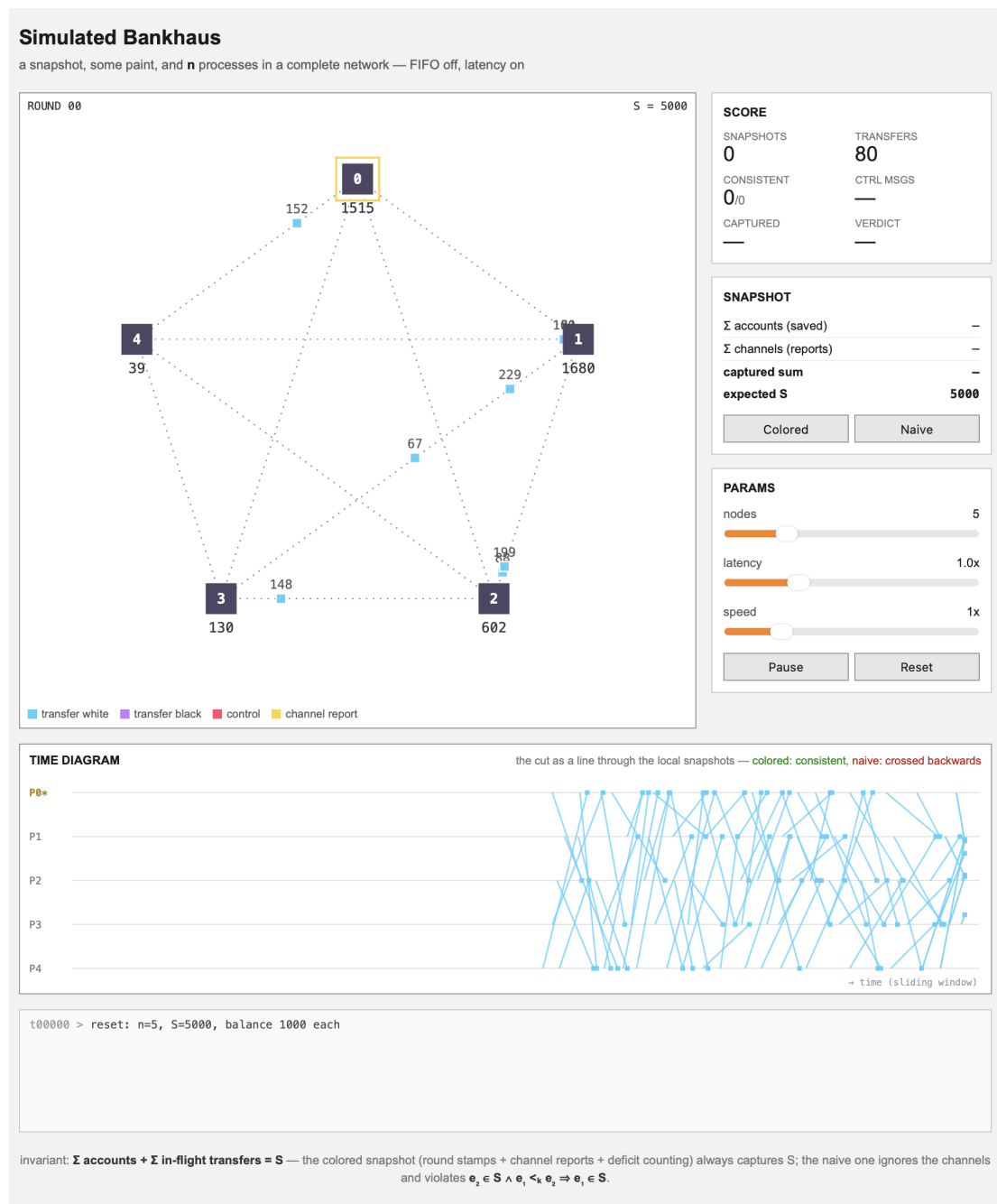


Abbildung 3: `visuals/index.html` im Browser (hier mit $n = 5$): links die Netzwerkansicht mit den überholenden Überweisungen, rechts Score, Schnappschuss-Auslöser und Parameter, unten das live mitgezeichnete Zeitdiagramm mit dem Schnitt durch die Prozess-Lebenslinien.

6 Fazit

Am Ende steht ein kleines, aber vollständiges Bild davon, was einen konsistenten Schnitt in einem realistischen – also nicht-FIFO-fähigen – Netz eigentlich ausmacht: Nicht die Ankunftsreihenfolge entscheidet, sondern eine Farbe, die schon beim Absenden feststeht; und nicht „alle Konten gleichzeitig lesen“, sondern auch die gerade unterwegs befindlichen Beträge mitzuzählen. Der naive Vergleich in Aufgabe 3 macht dabei greifbar, wie leicht Geld in einem verteilten System „entsteht“ oder „verschwindet“, sobald man diese Sorgfalt weglässt. Gleichzeitig zeigt die Statistik

aus Aufgabe 3 auch die Kehrseite des Einfärbeverfahrens: Der Koordinator bleibt ein zentraler Anlaufpunkt, und jeder Schnappschuss kostet spürbaren Zusatzverkehr – ein Kompromiss, den man in der Praxis gegen die Alternative (FIFO-Kanäle erzwingen und Chandy-Lamport verwenden) abwägen müsste.

Bekannte Einschränkung. Die Implementierung setzt, wie in Abschnitt 3 begründet, verlustfreie Kanäle und einen dauerhaft laufenden Koordinator voraus. Fällt P0 während eines laufenden Schnappschusses aus, terminiert dieser nie – ein neuer Schnappschuss könnte zwar von einem Ersatzkoordinator gestartet werden, der alte Zustand ginge aber verloren. Für den Rahmen dieser Übung ist das akzeptabel; in einem Produktivsystem bräuchte es entweder mehrere Koordinator-Kandidaten mit Übernahmelogik oder den Wechsel auf ein FIFO-basiertes Verfahren mit entsprechend höherem Aufwand beim Transport.